Software Requirements Specification (SRS) & Database Design

Library Management System

1. Introduction

This document outlines the Software Requirements Specification and Database Design for a Library Management System. The system manages book inventories, tracks student borrowing activities, and provides separate portals for Students and Administrators.

2. Technology Stack

- **Frontend**: React (with modern styling, hooks, context API or Redux)
- **Backend**: Spring Boot (Java, Spring Security, Spring Data JPA)
- **Database**: MySQL
- **Authentication**: JWT (JSON Web Tokens) for Stateless Session Management

3. User Roles & Access

- **Admin**: Manages books, monitors borrowing activities, downloads system reports.
- **Student**: Browses books, borrows books, sets return dates, and returns books.
- *Note: All endpoints (except Login and Registration) are heavily restricted and secured using JWT. Valid tokens are strictly validated against their roles (Student or Admin).*

4. Functional Requirements

4.1 Authentication Module

- **Registration**: Endpoints to register an account as either an Admin or a Student.
- **Login**: Validates credentials and responds with a JWT. Based on the role in the JWT, the application redirects users to their appropriate portals. There is no generic unauthenticated access beyond these two functions.

4.2 Student Portal

- **Book Dashboard**: Displays an interactive grid/list of all available books in the library.
- **Search & Filter Dropdowns**: Allows students to search by title/author, and provides a dropdown to sort and filter book categories based on their **Departments**.
- **Book Details Navigation**: Clicking on a specific book container navigates the user to a detailed view with a "Borrow" button.
- **Borrowing Flow**:

    - Clicking the "Borrow" button opens an input container (modal or dedicated form).
    - The student inputs an expected Return Date and Time.
    - Submitting the form finalizes the borrowing action and updates backend inventories.
- **Delayed Notifications**: Alerts the student (via UI banners or an unread notifications counter) if they have delayed their book submission past the return date/time.
- **Logout**: Ends the session and removes the JWT locally.

4.3 Admin Portal

- **Admin Dashboard (Home)**: Displays comprehensive metrics and lists all the books in the system.
- **Add Books Module**: Provides an input container form for the administrator to seamlessly add new book details (Title, Author, Department, Copies).
- **Live Borrowing Tracking (Table Format)**:
    - Once a student triggers a borrow action, it instantly populates in this table.
    - Table Columns: ID, Student Name, Department, Borrowed Book, Borrow Date/Time.
    - **Dynamic Status Indicators**:
        - 🟩 **Green (Returned)**: Indicates the student successfully returned the book.
        - 🟨 **Yellow (Pending)**: Indicates the book is currently borrowed and within the provided timeframe.
        - 🟥 **Red (Not Returned / Delayed)**: Indicates the projected return time has lapsed, but the book hasn't been submitted yet.
- **Report Generation**: Allows the Admin to download the current borrowing tracker report (as a CSV or PDF file).

---

5. Database Design (MySQL Schema)

5.1 Tables

1. users Table Stores the system's users and defines their authorization limits.

- id (INT, Primary Key, Auto Increment)

- full_name (VARCHAR)
- email (VARCHAR, Unique)
- password (VARCHAR, Bcrypt Hashed)
- role (ENUM: 'STUDENT', 'ADMIN')
- department (VARCHAR, Nullable - to track student departments)
- created_at (TIMESTAMP)

2. books Table Stores the library's catalog details.

- id (INT, Primary Key, Auto Increment)
- title (VARCHAR)
- author (VARCHAR)
- department (VARCHAR) - The book's category/department tag.
- total_copies (INT)
- available_copies (INT) - Decrements on borrow, increments on return.

3. borrowing_records Table Acts as the central transaction record tying Students to Books.

- transaction_id (INT, Primary Key, Auto Increment)
- student_id (INT, Foreign Key references users(id))
- book_id (INT, Foreign Key references books(id))
- borrow_date_time (DATETIME, default CURRENT_TIMESTAMP)
- expected_return_date_time (DATETIME)
- actual_return_date_time (DATETIME, Nullable)
- status (ENUM: 'PENDING', 'RETURNED', 'OVERDUE')

5.2 Entity-Relationship Flow

- A **User (Student)** can have multiple **Borrowing Records** (1:M).
- A **Book** can be associated with multiple **Borrowing Records** (1:M).
- When a student submits the borrow request, a borrowing_records entry is created with the status PENDING (Yellow).
- A cron job or JPA entity listener running in Spring Boot monitors if the expected_return_date_time is less than the current time; if so, its status shifts to OVERDUE (Red).

- When the student requests to return the book, actual_return_date_time is logged, available_copies augments, and the status locks to RETURNED (Green).

---

6. Architectural Logic & API Flow

Core API Endpoints (Spring Boot):

- POST /api/auth/register (Public)
- POST /api/auth/login (Public - Returns JWT)
- GET /api/books (Auth required - Supports ?department= filters)
- GET /api/books/{id} (Auth required)
- POST /api/books/add (Admin Only)
- POST /api/transactions/borrow (Student Only - Payload includes bookId, expectedReturnDate)
- PUT /api/transactions/return/{transactionId} (Student Only)
- GET /api/transactions/admin/view (Admin Only - fetches all transactions for the color-coded table)
- GET /api/transactions/admin/report/download (Admin Only - Response type text/csv)